

Meteora

DAMM v2 0.1.6

Smart Contract Security Assessment

December 2025

Prepared for:

Meteora

Prepared by:

Offside Labs

Sirius Xie





Contents

1	About Offside Labs	2
2	Executive Summary	3
3	Summary of Findings	6
4	Key Findings and Recommendations	7
4.1	Infinite Loop May Lead to DoS in swap IX	7
4.2	Conflict remaining_accounts Usage in intialize_reward IX	8
4.3	Improper Validation for Existing Pools in update_pool_fees IX	9
4.4	Informational and Undetermined Issues	10
5	Disclaimer	12



1 About Offside Labs

Offside Labs is a leading security research team, composed of top talented hackers from both academia and industry.

We possess a wide range of expertise in modern software systems, including, but not limited to, *browsers*, *operating systems*, *IoT devices*, and *hypervisors*. We are also at the forefront of innovative areas like *cryptocurrencies* and *blockchain technologies*. Among our notable accomplishments are remote jailbreaks of devices such as the **iPhone** and **PlayStation 4**, and addressing critical vulnerabilities in the **Tron Network**.

Our team actively engages with and contributes to the security community. Having won and also co-organized *DEFCON CTF*, the most famous CTF competition in the Web2 era, we also triumphed in the **Paradigm CTF 2023** within the Web3 space. In addition, our efforts in responsibly disclosing numerous vulnerabilities to leading tech companies, such as *Apple*, *Google*, and *Microsoft*, have protected digital assets valued at over **\$300 million**.

In the transition towards Web3, Offside Labs has achieved remarkable success. We have earned over **\$9 million** in bug bounties, and **three** of our innovative techniques were recognized among the **top 10 blockchain hacking techniques of 2022** by the Web3 security community.



<https://offside.io/>



<https://github.com/offsidelabs>



https://twitter.com/offside_labs



2 Executive Summary

Introduction

Offside Labs completed a security audit of DAMM v2 smart contracts, starting on November 27th, 2025, and concluding on December 9th, 2025.

Project Overview

The DAMM v2 Release 0.1.6 introduces two base-fee modes that decay as price rises—market-cap linear and market-cap exponential—adds an Operator account model with permission bits to update pool fee settings, and relaxes certain pool-creation token constraints. The default pool version is raised to 1, enabling higher fee ceilings for new pools; swap paths are optimized with Pinocchio to reduce CU. All administrative-style actions now require both a whitelisted address and operator.

Audit Scope

The assessment scope contains mainly the smart contracts of the cp-amm program for the DAMM v2 project.

The audit is based on the following specific branches and commit hashes of the codebase repositories:

- DAMM v2
 - Codebase: <https://github.com/MeteoraAg/damm-v2>
 - Release 0.1.6: [PR-124](#)
 - Commit Hash: 7147072caf16f3a37fa20f81f67986c629e33546

The issues described in this report have been resolved by the following commit:

- DAMM v2
 - Codebase: <https://github.com/MeteoraAg/damm-v2>
 - Commit Hash: 6b3dced7f6392190a4400e5ce56821b1b741a7f3

We listed the files we have audited below:

- DAMM v2 Release 0.1.6
 - programs/cp-amm/src/base_fee/base_fee_serde.rs
 - programs/cp-amm/src/base_fee/fee_market_cap_scheduler.rs
 - programs/cp-amm/src/base_fee/fee_rate_limiter.rs
 - programs/cp-amm/src/base_fee/fee_time_scheduler.rs
 - programs/cp-amm/src/base_fee/mod.rs
 - programs/cp-amm/src/constants.rs
 - programs/cp-amm/src/entrypoint.rs
 - programs/cp-amm/src/error.rs
 - programs/cp-amm/src/event.rs
 - programs/cp-amm/src/instructions/admin/auth.rs



- programs/cp-amm/src/instructions/admin/ix_close_operator_account.rs
- programs/cp-amm/src/instructions/admin/ix_create_operator_account.rs
- programs/cp-amm/src/instructions/admin/mod.rs
- programs/cp-amm/src/instructions/initialize_pool/ix_initialize_customizable_pool.rs
- programs/cp-amm/src/instructions/initialize_pool/ix_initialize_pool.rs
- programs/cp-amm/src/instructions/initialize_pool/ix_initialize_pool_with_dynamic_config.rs
- programs/cp-amm/src/instructions/ix_add_liquidity.rs
- programs/cp-amm/src/instructions/ix_create_position.rs
- programs/cp-amm/src/instructions/ix_fund_reward.rs
- programs/cp-amm/src/instructions/ix_remove_liquidity.rs
- programs/cp-amm/src/instructions/mod.rs
- programs/cp-amm/src/instructions/operator/ix_claim_protocol_fee.rs
- programs/cp-amm/src/instructions/operator/ix_close_config.rs
- programs/cp-amm/src/instructions/operator/ix_close_token_badge.rs
- programs/cp-amm/src/instructions/operator/ix_create_dynamic_config.rs
- programs/cp-amm/src/instructions/operator/ix_create_static_config.rs
- programs/cp-amm/src/instructions/operator/ix_create_token_badge.rs
- programs/cp-amm/src/instructions/operator/ix_initialize_reward.rs
- programs/cp-amm/src/instructions/operator/ix_set_pool_status.rs
- programs/cp-amm/src/instructions/operator/ix_update_pool_fees.rs
- programs/cp-amm/src/instructions/operator/ix_update_reward_duration.rs
- programs/cp-amm/src/instructions/operator/ix_update_reward_funder.rs
- programs/cp-amm/src/instructions/operator/mod.rs
- programs/cp-amm/src/instructions/swap/ix_p_swap.rs
- programs/cp-amm/src/instructions/swap/ix_swap.rs
- programs/cp-amm/src/instructions/swap/mod.rs
- programs/cp-amm/src/instructions/swap/swap_exact_in.rs
- programs/cp-amm/src/instructions/swap/swap_exact_out.rs
- programs/cp-amm/src/instructions/swap/swap_partial_fill.rs
- programs/cp-amm/src/lib.rs
- programs/cp-amm/src/macros.rs
- programs/cp-amm/src/math/fee_math.rs
- programs/cp-amm/src/math/safe_math.rs
- programs/cp-amm/src/params/fee_parameters.rs
- programs/cp-amm/src/state/config.rs
- programs/cp-amm/src/state/fee.rs
- programs/cp-amm/src/state/mod.rs
- programs/cp-amm/src/state/operator.rs
- programs/cp-amm/src/state/pool.rs
- programs/cp-amm/src/utils/mod.rs
- programs/cp-amm/src/utils/p_helper.rs
- programs/cp-amm/src/utils/token.rs



Findings

The security audit revealed:

- 0 critical issue
- 1 high issue
- 1 medium issue
- 1 low issue
- 5 informational issues

Further details, including the nature of these issues and recommendations for their remediation, are detailed in the subsequent sections of this report.



3 Summary of Findings

ID	Title	Severity	Status
01	Infinite Loop May Lead to DoS in swap IX	High	Fixed
02	Conflict remaining_accounts Usage in initialize_reward IX	Medium	Fixed
03	Improper Validation for Existing Pools in update_pool_fees IX	Low	Fixed
04	Inconsistent ErrorCode Between p_load_mut_checked and p_load_mut_unchecked	Informational	Fixed
05	Incorrect Event Vaule for EvtCreateClaimFeeOperator.operator	Informational	Fixed
06	Typo	Informational	Fixed
07	Missing Validation for number_of_period in MarketCapScheduler	Informational	Fixed
08	Improper ErrorCode in p_event_dispatch	Informational	Fixed



4 Key Findings and Recommendations

4.1 Infinite Loop May Lead to DoS in swap IX

Severity: High

Status: Fixed

Target: Smart Contract

Category: DoS

Description

In `validate_single_swap_instruction`, it checks whether the instruction's accounts contain the pool account.

```
for i in 0..current_index {
    ...
    if instruction.get_program_id() != crate::ID.as_array() {
        // we treat any instruction including that pool address is
        // other swap ix
        loop {
            match instruction.get_account_meta_at(i.into()) {
                Ok(account_metadata) => {
                    if &account_metadata.key == pool.as_array() {
                        msg!("Multiple swaps not allowed");
                        return Err(PoolError::
                            FailToValidateSingleSwapInstruction.into());
                    }
                }
                ...
            }
        }
    }
}
```

[programs/cp-amm/src/instructions/swap/ix_p_swap.rs#L315-L338](#)

In the inner loop, however, there is no update to the loop variable, and it directly uses the outer loop variable `i`. Once the validation logic enters the inner loop, it will fall into an infinite loop.

Moreover, this validation is also incomplete, as it only validates the `i-th` `AccountMeta` of the `i-th` instruction and ignores the other `AccountMeta`.

Impact

For a transaction where the swap is not the first IX, it may fall into this infinite loop until the transaction's compute units are exhausted and it fails; this issue will directly cause users to be unable to perform swaps.



Recommendation

It is recommended to distinguish the loop counters of this nested loop and use a different variable (for example, `j`) to iterate over the instruction's `AccountMeta`.

Mitigation Review Log

Fixed in the commit `6b3dced7f6392190a4400e5ce56821b1b741a7f3`.

4.2 Conflict `remaining_accounts` Usage in `initialize_reward` IX

Severity: Medium

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

In the `initialize_reward` IX, there are two places where `remaining_accounts[0]` is used.

TokenBadge:

```
74         is_token_badge_initialized(  
75             ctx.accounts.reward_mint.key(),  
76             ctx.remaining_accounts  
77                 .get(0)  
78                 .ok_or(PoolError::InvalidTokenBadge)?  
79         )?,
```

[programs/cp-amm/src/instructions/operator/ix_initialize_reward.rs#L74-L79](#)

Operator Account:

```
1274         let operator_account = remaining_accounts  
1275             .get(0)  
1276             .ok_or_else(|| PoolError::InvalidAuthority)?;
```

[programs/cp-amm/src/state/pool.rs#L1274-L1276](#)

When `reward_mint` is not a supported mint, `remaining_accounts[0]` becomes ambiguous because it is required in both places at the same time.

Impact

As a result, the operator cannot use `initialize_reward` to create a corresponding reward for an unsupported mint with a `TokenBadge`.



Recommendation

It is recommended to adjust the usage of `remaining_accounts` in this instruction.

Mitigation Review Log

Fixed in the commit 6b3dced7f6392190a4400e5ce56821b1b741a7f3.

4.3 Improper Validation for Existing Pools in `update_pool_fees` IX

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

The `update_pool_fees` IX can update `cliff_fee_numerator` in the base fee, and it performs a validation after the update.

However, during this validation, it basically only requires `max_fee_numerator <= get_max_fee_numerator(CURRENT_POOL_VERSION)`, and `CURRENT_POOL_VERSION` is changed from 0 to 1 in this PR, which means the upper bound of `max_fee_numerator` changes from `500_000_000` to `990_000_000`.

For pools where `pool.version == 0`, the upper bound of `max_fee_numerator` should still be `500_000_000`.

Impact

This may cause operators to use the `update_pool_fees` IX to set the `cliff_fee_numerator` of some version 0 pools to abnormal values. Although `max_fee_numerator` is restricted according to `pool.version` during the swap, it is still possible here to set a `cliff_fee_numerator` value that does not comply with the V0 requirements.

Recommendation

It is recommended to add a `pool_version` parameter to `BaseFeeHandler.validation` to distinguish the different `max_fee_numerator` values for pools of different versions via this parameter.

Mitigation Review Log

Fixed in the commit 6b3dced7f6392190a4400e5ce56821b1b741a7f3.



4.4 Informational and Undetermined Issues

Inconsistent ErrorCode Between p_load_mut_checked and p_load_mut_unchecked

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Code QA

In `p_load_mut_checked` and `p_load_mut_unchecked`, the error codes when calling `acc_info.try_borrow_mut_data()` are different:

`p_load_mut_checked`:

```
73     let mut data = acc_info
74         .try_borrow_mut_data()
75         .map_err(|_| ProgramError::AccountBorrowFailed)?;
```

[programs/cp-amm/src/utils/p_helper.rs#L73-L75](#)

`p_load_mut_unchecked`:

```
90     let mut data = acc_info
91         .try_borrow_mut_data()
92         .map_err(|err| ProgramError::from(u64::from(err)))?;
```

[programs/cp-amm/src/utils/p_helper.rs#L90-L92](#)

It is recommended to keep the errors consistent between the two functions.

Incorrect Event Vaule for EvtCreateClaimFeeOperator.operator

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Code QA

In the `create_claim_fee_operator` IX, the final emitted event `EvtCreateClaimFeeOperator` sets `operator` to `ctx.accounts.operator.key()`, but it should be set to `ctx.accounts.claim_fee_operator.key()`.

Typo

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Code QA

The formula written in the comment is incorrect:

```
56     // passed_period = (current_sqrt_price - init_sqrt_price) * 10_000 *
57     //     init_sqrt_price / price_step_bps
58     FeeMarketCapSchedulerLinear,
```

[programs/cp-amm/src/state/fee.rs#L56-L57](#)

The intended formula here should be:



```
passed_period = (current_sqrt_price - init_sqrt_price) * 10_000 /  
init_sqrt_price / price_step_bps
```

Missing Validation for number_of_period in MarketCapScheduler

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Data Validation

In `PodAlignedFeeMarketCapScheduler.validate`, it is explicitly required that `reduction_factor`, `price_step_bps`, and `scheduler_expiration_duration` must be greater than 0. However, there is no restriction on `number_of_period`. If `number_of_period == 0`, then the other parameters are meaningless regardless of whether they are greater than 0, and `base_fee_numerator` will always be `cliff_fee_numerator`. It is recommended to also add a constraint `number_of_period > 0`.

Improper ErrorCode in p_event_dispatch

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Code QA

In `p_event_dispatch`, it first checks whether `given_event_authority` is a signer, and if not, it returns `ErrorCode::ConstraintSeeds`. However, using Anchor's existing `ErrorCode::ConstraintSigner` would be more appropriate here.



5 Disclaimer

This audit report is provided for informational purposes only and is not intended to be used as investment advice. While we strive to thoroughly review and analyze the smart contracts in question, we must clarify that our services do not encompass an exhaustive security examination. Our audit aims to identify potential security vulnerabilities to the best of our ability, but it does not serve as a guarantee that the smart contracts are completely free from security risks.

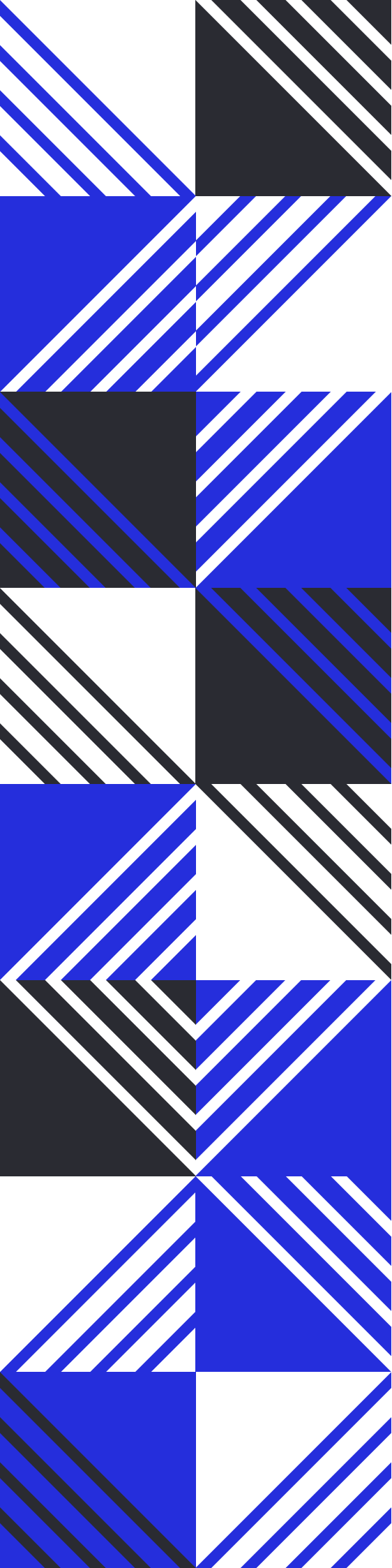
We expressly disclaim any liability for any losses or damages arising from the use of this report or from any security breaches that may occur in the future. We also recommend that our clients engage in multiple independent audits and establish a public bug bounty program as additional measures to bolster the security of their smart contracts.

It is important to note that the scope of our audit is limited to the areas outlined within our engagement and does not include every possible risk or vulnerability. Continuous security practices, including regular audits and monitoring, are essential for maintaining the security of smart contracts over time.

Please note: we are not liable for any security issues stemming from developer errors or misconfigurations at the time of contract deployment; we do not assume responsibility for any centralized governance risks within the project; we are not accountable for any impact on the project's security or availability due to significant damage to the underlying blockchain infrastructure.

By using this report, the client acknowledges the inherent limitations of the audit process and agrees that our firm shall not be held liable for any incidents that may occur subsequent to our engagement.

This report is considered null and void if the report (or any portion thereof) is altered in any manner.



 <https://offside.io/>

 <https://github.com/offsidelabs>

 https://twitter.com/offside_labs